Akamai adquirirá LayerX para reforzar el control de uso de IA en cualquier navegador. Obtener detalles

¿Por qué Akamai?
Back ¿Por qué Akamai?
Close

Productos
Back Productos Close
Cloud computing
Ciberseguridad
Distribución de contenido
Servicios globales
Todos los productos

Soluciones
Back Soluciones Close
Inteligencia artificial (IA)
Akamai Inference Cloud
Almacenamiento
Block Storage
Object Storage
Backups
Bases de datos
Managed Databases
Recursos informáticos
GPU
CPU
Accelerated Compute
Kubernetes
App Platform
Serverless
Akamai Functions
Precios
Lista de precios
Redes
Cloud Firewall
DNS Manager
NodeBalancers
Cloud computing
Serverless
Medios
SaaS
Videojuegos
Seguridad
Cumplimiento relacionado con la ciberseguridad
Protección contra ransomware
Aplicaciones y API seguras
Distribución y seguridad de DNS
Zero Trust
Protección contra DDoS
Protección contra los bots y abusos
Gestión de acceso, credenciales e

Recursos
Back Recursos Close
Aprenda
White papers, libros electrónicos, vídeos, informes sobre productos
Historias de clientes
Formación y certificación
Investigación sobre ciberseguridad
Grupo de inteligencia sobre seguridad de Akamai
Informes sobre el estado de Internet (SOTI)
desarrolladores de nube
Centro de desarrolladores
Guías y tutoriales

Partners
Back Partners Close
Partners de canal
Resumen del programa
Únase a Akamai Partner Connect
Recursospara partners
Partner Portal
Partners tecnológicos
Resumen del programa
Encontrar un partner
Directorio de partners del canal
Directorio de partners tecnológicos
Historia de partner
Cloud

Akamai Cloud
Cree aplicaciones de baja latencia impulsadas por IA con nuestra nube de código abierto y asequible.
[
Ver precios

Akamai Security
Proteja sus aplicaciones y redes con inteligencia en tiempo real y IA adaptable.
[
Ver precios

Nuestr

Back Ciberseguridad Close
de
Rendimiento
respuestas de la
Partner Referrals

Back

a infraestructura global

Acerca las experiencias de IA a los usuarios con la plataforma cloud y la red edge más distribuidas.

Akamai Cloud

Akamai Security

Nuestra infraestructura global

Distribución de contenido Cloud

aplicaciones y API

Distribución de contenido multimedia

Edge Computing

Soluciones del sector

Medios de comunicación y entretenimiento

Retail, turismo y hostelería

Servicios financieros

Salud y ciencias de la vida

sector público

Defensa

Juegos

Proveedores de servicios

Comunidad

Programa beta

Programas para startups

Novedades

Blog de Akamai

Eventos y talleres

Iniciar sesión

Probar Akamai

¿Bajo ataque?

Análisis de las amenazas de autenticación de usuarios comprometida para JSON Web Tokens



Nitzan Namer

June 07, 2023



Resumen ejecutivo

- Los investigadores de Akamai han analizado los JSON Web Tokens (JWT) como vector de ataques de autenticación de usuarios comprometida, que se encuentran en el Proyecto Abierto de Seguridad de Aplicaciones Web (OWASP), entre los diez principales riesgos de seguridad de API,y descubrió diferentes escenarios en los que se dan las amenazas y tendencias de JWT.
- Los JWT son responsables de proteger las API mediante la emisión de tokens (normalmente entre clientes y servidores) para verificar de forma segura a los usuarios. Estos tokens son uno de los formatos de verificación más comunes que se utilizan y contienen información que se comparte en forma de objetos JSON.
- Aunque no todos los tokens están cifrados, sí están codificados y tienen una firma de verificación. Examinamos algunas de las muchas amenazas a los JWT que se deben a una implementación incorrecta o a claves secretas cortas y de entropía baja.
- El robo de cuentas, la derivación de privilegios y las filtraciones de datos son tres grandes riesgos potenciales a los que se enfrentan los usuarios y las organizaciones de JWT al elegir un JWT como token en una API. Observamos en nuestro tráfico que la mayoría de los JWT utilizan un algoritmo simétrico, aunque teóricamente es menos seguro.
- En esta publicación, ofrecemos prácticas recomendadas sobre cómo gestionar estas amenazas a los JWT.

Síganos en Twitter

Introducción

Los JWT son un esquema de tokens ampliamente utilizado y relativamente fácil de implementar que oculta muchos riesgos. Estos riesgos deben tenerse en cuenta al implementar y asegurar los JWT. Uno de los riesgos más importantes para los JWT, que aparece entre los 10 principales riesgos de seguridad de API de la OWASP, es la autenticación de usuario comprometida.

La autenticación de usuario comprometida se produce cuando una API no verifica correctamente las credenciales y la identidad del usuario que realiza la solicitud. Lamentablemente, se trata de una vulnerabilidad común observada en los JWT y puede provocar que un atacante suplante a un usuario o acceda a la cuenta de otro usuario, permitiendo la derivación de privilegios y la filtración de datos.

Protección del canal

El JWT es un token de autenticación utilizado habitualmente en las API y las aplicaciones web. En nuestro tráfico observamos que las aplicaciones web están adoptando más API y exponiéndolas a los usuarios, lo que plantea la necesidad de una autenticación perfecta en diferentes terminales de las API.

La API es el canal principal de una organización y contiene los mecanismos de autenticación y autorización y consulta de bases de datos y expone la mayoría de las funciones de la organización. Por ese motivo, las amenazas a los JWT son más peligrosas en el ámbito de las API. En esta entrada del blog, vamos a introducir los conceptos básicos de los JWT, seis amenazas a los JWT y cómo abordarlas.

Conceptos básicos de los JWT

El JWT es un formato para enviar datos JSON firmados con el fin de transmitir datos en aplicaciones web, aplicaciones móviles y API, que se utiliza principalmente con fines de autenticación. La estructura de los JWT consta de tres elementos: el encabezado (también conocido como encabezado JOSE), la carga y la firma, todos separados por un punto (Figura 1).

```
eyJ0eXAiOiJKV1QiLCJhbGciOiJIUzI1NiJ9.eyJpc3MiOiIiLCJpYXQiOjE2...
```

Copy

Fig. 1: Este es un ejemplo de los tres elementos de la estructura de los JWT

El encabezado y la carga

Tanto el encabezado como la carga tienen codificación Base64, con seguridad para las URL, y recomendaciones de especificaciones IETF para los campos (Figura 2). El hecho de que tengan “codificación” significa que no son directamente legibles por el ojo humano, pero el servidor sí los descodifica fácilmente. El objetivo es asegurar la integridad y usabilidad de los

datos y la “seguridad para las URL”. De esta forma, no tendrán caracteres superpuestos que utilicen en los navegadores web.

El encabezado JOSE se utiliza tanto para el cifrado web JWT y JSON (JWE), que es básicamente un JWT cifrado que ayuda a evitar los ataques a los JWT más comunes. El campo que especifica el método de token elegido es “typ” (typ:JWT/JWE). La carga puede contener campos registrados (reclamados por la Autoridad de Números Asignados de Internet) o campos personalizados, dependiendo de la implementación.

Fig. 2: JWT codificado

La firma

El propósito de la firma es verificar el token, lo que significa que el servidor forjó el token y que los datos no han cambiado desde que se creó el token.

Existen dos pasos principales para crear la firma:

1. Aplicación de un algoritmo de cifrado (por ejemplo, MAC) en el encabezado y la carga, separado por un punto (el algoritmo utiliza una clave secreta)
2. Aplicación de un algoritmo hash (por ejemplo, SHA256) en el encabezado cifrado y la carga

Cuando se utiliza un JWT, el servidor verifica la firma para validar el propio token. (Más adelante explico los diferentes algoritmos que se utilizan para la validación).

Los JWT se utilizan ampliamente para la autenticación en aplicaciones web debido a su facilidad de uso (incluso a gran escala) y sencillez de implementación, y porque los datos necesarios del servidor se guardan en el lado del cliente. Sin embargo, existen riesgos asociados con los JWT aunque estén firmados. JWT utiliza texto sin formato y cada implementación consta de diferentes campos de carga. Por lo tanto, hay una amplia superficie de ataque y mucho espacio para errores.

Al explorar algunas de las amenazas más comunes e interesantes a las que se enfrentan los JWT, podemos ser más conscientes de las vulnerabilidades, detectar con mayor facilidad comportamientos maliciosos, mitigar mejor los riesgos y preparar las medidas de seguridad necesarias para protegernos frente a posibles vulnerabilidades.

Nuestro objetivo es proporcionar a los usuarios un enfoque más seguro al trabajar con JWT y a los profesionales y administradores de seguridad las herramientas y recomendaciones necesarias para ayudar a cumplir con su diligencia debida.

Seis amenazas a los JWT

1. Permitir que el servidor utilice un token sin validación

Dado que los JWT están firmados en lugar de cifrados, la validación se debe realizar antes de cualquier uso. En el escenario de amenaza más básico, en el que una aplicación no realiza la validación en absoluto, un atacante puede editar la carga (por ejemplo, derivación de privilegios)y mantener la firma intacta, o incluso eliminarla y obtener permisos superiores.

Otro método es utilizar el parámetro “alg” en el encabezado, que representa el algoritmo que se ha utilizado para firmar el token. “None” es un valor legítimo (y el JWT se denomina JWT no seguro), por lo que cualquiera puede firmar fácilmente el token. En el back-end, existe también una posible implementación de la verificación de JWT.

Este enfoque comienza buscando el campo “alg” en el encabezado. Después, se usa el algoritmo especificado para verificar el token. El algoritmo “None” representa que no existe necesidad de firma y verificación para ese token. La edición de la carga, el encabezado (a alg:none) y la “firma automática” (eliminando la firma) del token llevará a un ataque exitoso.

Ejemplo

La figura 3 muestra un ejemplo de un JWT descodificado.

```
eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9.eyJuYW1lIjoiZXhhbXBsZSBu...
oxNjU5MjM5MDIyfQ.PrhAp2DUWXoL01_odyLATuzrwn5rMtY1IVsP8y4LH5E
decoded:
{
  "alg": "HS256",
  "typ": "JWT"
}
.{
  "name": "example name",
  "iat": 1659239022
}
```

Fig. 3: Ejemplo de JWT descodificado

Asumimos que el terminal de la API utiliza el campo “alg” en la carga como algoritmo elegido para la verificación.

Pero nos gustaría editar el JWT (Figura 4). De esta manera, logramos generar un JWT válido de otro usuario.

```
eyJhbGciOiJub25lIiwidHlwIjoiSldUIn0.eyJuYW1lIjoiZXhhbXBsZSBu...
xNjU5MjM5MDIyfQ.
decoded:
{
  "alg": "none",
  "typ": "JWT"
}
.{
  "name": "other person's name",
```

```
  "iat": 1659239022
}
```

Fig. 4: JWT editado para explotar alg:none.

Prácticas recomendadas: Esta es fácil: **siempre** hay que realizar la validación con un algoritmo predefinido integrado antes de utilizar el token. No se puede utilizar el parámetro alg como puntero para el algoritmo de verificación.

2. Utilizar la misma clave privada para diferentes aplicaciones

Hay muchas empresas que optan por tener un registro separado para las distintas aplicaciones. El uso de la misma clave privada creará un token válido en ambas aplicaciones, pero con un ID de usuario diferente. En ese caso, se trata de realizar una solicitud a una segunda aplicación con el token de aplicación de la primera y apropiarse de una cuenta (no elegida) (Figura 5).

Fig. 5: Uso del JWT de una segunda aplicación en otra aplicación, que provoca un robo de cuentas

Prácticas recomendadas: Esta es otra regla básica: al separar ID de usuario, hay que usar claves privadas diferentes.

3. Uso de un algoritmo de firma débil

Existen dos familias de algoritmos principales: simétricos (HSXXX) y asimétricos (RSXXX,ESXXX,PSXXX). Un algoritmo simétrico requiere una clave secreta compartida; un algoritmo asimétrico requiere claves privadas y públicas (Tabla 1).

Simétrico			
HSXXX	HMAC + SHA	Algoritmo relativamente débil	Utiliza c compar
Asimétrico			
RSXXX	RSA + SHA	Algoritmo más seguro	Utiliza c y públic
ESXXX	ECDSA + SHA	El algoritmo más	Utiliza c

		seguro de los tres	y públic

Tabla 1: Las dos principales familias de algoritmos

Desde el punto de vista del atacante, un algoritmo simétrico es más fácil de forzar, no es necesario recopilar claves públicas y el tiempo de computación es lo más eficiente posible. Hemos observado en nuestro tráfico que los algoritmos simétricos se utilizan más que los algoritmos asimétricos (Figura 6).

Teóricamente, los algoritmos simétricos requieren menos computación, por lo que a gran escala podrían tener un mejor rendimiento que los asimétricos.

Fig. 6: Un estudio de los algoritmos de JWT en el tráfico de Akamai muestra que los algoritmos simétricos son los más comunes

Prácticas recomendadas: Utilizar algoritmos asimétricos. Recomendamos utilizar el algoritmo ECDSA, que actualmente es la opción más segura (criptográficamente). Una clave simétrica bien asegurada resulta aceptable si la clave secreta es de entropía alta y lo suficientemente larga.

4. Elegir una clave privada de entropía corta o baja

Una clave privada de entropía corta o de baja se podría forzar rápidamente y a bajo coste. Existe un software de procesamiento fácil de usar que se puede instalar en entornos en la nube de alto rendimiento con unidades de procesamiento de gráficos (GPU). El rendimiento de las GPU está creciendo con rapidez y eso repercute en la capacidad para descifrar claves. Las tarjetas gráficas de gama alta de hoy en día funcionan casi 2,5 veces más rápido que hace tres años.

En la Tabla 2 se muestra una comparación de cuántos días se tardaría en descifrar una clave privada de ocho caracteres (independientemente de la entropía) hace tres años y hoy.

Tarjeta gráfica de gama alta	**Días para descifrado**
Hace tres años	10,5 días
Hoy	4,5 días

Tabla 2: El número de días que se tardaría en descifrar una clave privada de ocho caracteres

Una clave privada de siete caracteres que habría tardado unas 24 horas en descifrarse hace unos años hoy se puede descifrar en menos de 90 minutos. La singularidad de este ataque es

que se ejecuta sin necesidad de conexión, sin ninguna posibilidad de defensa por parte de la organización o incluso sin que sepa que la están atacando.

Imagínese a sí mismo como un atacante patrocinado por el estado que crea un sistema coordinado para ello. Mantener la clave privada del JWT significa que el atacante pueda generar un token firmado y suplantar a cualquier otro usuario simplemente conociendo su ID.

Prácticas recomendadas: Elegir una clave privada con entropía larga (al menos 10 caracteres) y alta. Recomendamos utilizar RSA como generador secreto.

5. Mantener datos confidenciales en la carga de un JWT

Los JWT están codificados con Base64 URL, por lo que la carga no está cifrada. El almacenamiento de nombres de bases de datos, ID incrementales (de todo tipo) o cualquier campo y datos internos del servidor crea una exposición excesiva de los datos, lo que puede ayudar a un atacante en otros vectores de ataque (por ejemplo, autorización a nivel de objeto comprometida).

Prácticas recomendadas: No almacenar datos confidenciales en un JWT. Si hace falta almacenar datos confidenciales, utilizar JWE.

6. Confundir las claves

En los algoritmos asimétricos, se utiliza una clave privada para forjar el JWT (la parte de la firma) y una clave pública para validar el JWT. Un atacante puede cambiar el algoritmo especificado en el encabezado por uno simétrico (por ejemplo, HS256) y crear una firma utilizando la clave pública que el servidor utiliza legítimamente para la verificación.

En ese escenario, el back-end no implementado correctamente utilizará la clave pública y ejecutará el algoritmo simétrico (como el atacante especificado en el encabezado) para verificar correctamente el JWT generado por el atacante.

Ejemplo

La figura 7 muestra el proceso del servidor de generar un nuevo JWT y la figura 8 ilustra el proceso de verificación de JWT en el servidor.

Fig. 7: El proceso de generación de JWT en el servidor

Fig. 8: El proceso de validación de JWT en el servidor

En la última parte de la verificación, puede producirse un error en la comparación por una de las dos razones (o ambas):

1. El encabezado y la carga se editaron, pero la firma no
2. Hay una configuración incorrecta del algoritmo o la clave correcta

La figura 9 muestra un ataque de confusión de clave.

Fig. 9: Proceso de falsificación de JWT malicioso de un atacante que incluye confusión de claves

El paso final: El servidor verificará el token con el proceso ilustrado en la figura 8, en el que el servidor confía en el campo alg. Verificará el token ejecutando el algoritmo HS256 con la clave pública como secreta, que es exactamente lo que planeó el atacante.

Prácticas recomendadas: utilice algoritmos predefinidos en el proceso de verificación y no confíe en la introducción por parte de los usuarios.

Conclusión

JWT es el token de autenticación más común. Permite el uso de muchas aplicaciones web y API mientras se autentica, sin necesidad de tener que iniciar sesión con frecuencia. Sin embargo, los JWT implican muchas amenazas porque no están cifrados ni se implementan teniendo en cuenta la seguridad.

- La potencia de los ordenadores está evolucionando de manera rápida y eso pone en riesgo las claves secretas débiles y lograr romperlas en días en lugar de años. En el caso de los JWT, este tipo de ataque puede ejecutarse sin necesidad de conexión y que una organización no tenga conocimiento de los ataques.
- Los algoritmos simétricos son el tipo de algoritmo más común y simplemente facilitan la vida de los atacantes.
- Existen otras amenazas más complejas, como los ataques de autenticación, pero para los atacantes resulta difícil automatizar los ataques de autenticación. Sin embargo, los vectores de ataque JWT simples se pueden automatizar y explotar masivamente.

Por estas razones (y muchas otras), resulta crucial que los usuarios sean conscientes de las vulnerabilidades de los JWT y conozcan las mejores prácticas de seguridad que deben implementar para protegerse contra posibles vulnerabilidades. Esperamos que este análisis sirva de guía práctica para proteger los JWT de la autenticación de usuarios comprometida, uno de los diez principales riesgos de seguridad de API de la OWASP.

Consulte la investigación con más detalle

Ciberseguridad Investigación

Inteligencia ante amenazas

Investigaciones sobre seguridad



escrito por
Nitzan Namer

Nitzan Namer es experto en seguridad en Akamai.

Entradas de blog relacionadas

CIBERSEGURIDAD

CVE-2026-31979: la trampa de Symlink: derivación de privilegios root en Himmelblau

March 20, 2026

Una vulnerabilidad de gravedad alta (CVE-2026-31979) afecta a determinadas implementaciones de Himmelblau. Se recomienda tomar medidas inmediatas.

por Akamai Security Intelligence Group

Más información >

CIBERSEGURIDAD

Akamai ayuda a las autoridades a interrumpir las botnets del IoT más grandes del mundo

March 19, 2026

Recientemente, el Departamento de Justicia de EE. UU. ha interrumpido varias botnets de DDoS potentes y de gran tamaño y ha cerrado sus servicios de DDoS de alquiler con la ayuda de Akamai.

por Akamai Security Intelligence Group

Más información >

INVESTIGACIONES SOBRE SEGURIDAD

IA en la detección de vulnerabilidades: una llamada a la supervisión y la precaución humanas

March 13, 2026

Descubra por qué la supervisión humana es crucial para el uso responsable de los sistemas de IA a la hora de identificar vulnerabilidades de seguridad y saber cómo evitar falsos positivos.

por Larry Cashdollar y Kyle Lefton

Más información >

Valore la utilidad de esta página ☆ ☆ ☆ ☆ ☆

©2026 Akamai Technologies